Reusable Reporting-Unit Lineage Packages

Gio Della-Libera

Draft — June 28, 2026

Abstract

Precinct and reporting-unit history is a shared public-evidence dimension, not a private attachment to one plan or count package. RHIST makes that dimension a small package: cycles name the time slices, context indexes name the units available in each cycle, lineage events declare unchanged, rename, split, and merge relationships, and crosswalk rows carry rational weights between unit sets. The implemented verifier checks source references, cycle and context links, lineage unit references, split/merge/rename cardinality, exhaustive crosswalk weight sums, claim-boundary presence, and a canonical package content hash. RCOUNT can map its existing lineage records into RHIST events and can declare RHIST package-hash references, so counts and plans can point to a stable lineage substrate without absorbing ownership of it.

1 Introduction

Precinct and reporting-unit histories recur across election evidence systems. A count package needs to say why a precinct total in one election cannot be joined directly to a later precinct id. A plan package needs to compare assignments across census or administrative cycles. Neither package should permanently embed the whole unit-history substrate.

RHIST therefore treats reporting-unit lineage as a reusable package. RPLAN and RCOUNT can reference the same lineage artifact by content hash, while RHIST owns the narrower question of which units existed in which cycles and how declared unit identities changed between those cycles. The package is small enough to audit directly and stable enough to be reused by later count, context, and plan artifacts.

This paper documents the first implemented RHIST slice. It does not assert that any jurisdiction drew legally correct boundaries or that every historical source is complete. It claims only that the declared cycle, unit, lineage, and crosswalk records are replayable from a package directory and that downstream packages can bind to that replayable lineage by hash.

2 Lineage Contract

The core package type is `RhistPackage`. Its manifest declares the RHIST version, package id, jurisdiction, cycle ids, producer, creation time, and `package_content_hash`. The unit surface

is represented by `ContextIndexEntry`: each context has a `context_hash`, an `rctx_version`, a unit kind, a cycle id, and the unit ids available in that cycle. `CycleRecord` then binds each cycle to one context id and context hash.

Record or field	Meaning
`CycleRecord`	Cycle id, jurisdiction, `CycleKind`, effective date, context id, context hash, and source refs.
`ContextIndexEntry.unit_ids`	Reporting-unit identities available for one cycle and unit kind.
`LineageEvent`	Event id, `LineageEventKind`, from and to cycles, from and to unit ids, effective date, authority, confidence, source refs, and explanation.
`CrosswalkRecord`	From and to cycles, context hashes, unit ids, rational `weight`, `CrosswalkWeightKind`, exhaustive flag, and source refs.
`SourceIndexEntry`	Source id, package-relative source path, SHA-256 digest, media type, and description.
`ClaimBoundary`	Package id, what the package proves, what it does not prove, and caveats.

The implemented package directory mirrors those records: `manifest.json`, `sources/source-index.json`, `contexts/context-index.ndjson`, `units/cycles.ndjson`, `units/lineage-events.ndjson`, optional `units/crosswalks.ndjson`, and `claims/claim-boundary.json`. The writer also emits `proofs/package-hashes.json` and `transcripts/verify-transcript.json`.

The package hash is canonical. `package_content_hash` removes the declared hash from the manifest, serializes the remaining package fields in canonical JSON order, prefixes the bytes with `RHIST_PACKAGE_V1`, and returns a `sha256:` digest. Source bytes are separately checked against the source index by `rhist-io`.

3 Implemented Surface

`rhist-core` owns record validation. `verify_package` checks the manifest cycle references, source reference resolution, context cycle links, unique context unit ids, cycle-to-context links, lineage unit references, lineage cardinality, crosswalk unit references, exhaustive crosswalk weight sums, and claim-boundary presence. Cardinality is type-specific: `Rename`, `Unchanged`, and `AdministrativeRecode` are one-to-one; `Split` is one-to-many; `Merge` is many-to-one; `Create`, `Close`, and `BoundaryChange` have their own declared shapes.

`rhist-io` owns package directories. `read_package_dir` loads the manifest, source index, contexts, cycles, lineage events, optional crosswalks, and claim boundary; verifies source files; verifies the package hash; and then calls the core verifier. `write_package_dir` emits the same directory shape, including `proofs/package-hashes.json`. `refresh_package_hashes` regenerates computed package hashes for fixtures.

`rhist-cli verify` is the command surface. It reads a package directory through `rhist-io`, returns `pass` or `fail`, and can write a JSON or pretty JSON transcript containing the package id, computed package content hash, check ids, and error text.

Check	Failure caught
manifest_cycle_refs	Unsupported RHIST version, empty package id, invalid or mismatched package hash, and manifest cycles missing from cycle records.
source_refs_resolve	Duplicate or invalid source ids and source references not declared in the source index.
context_cycle_refs and context_unit_ids_unique	Contexts pointing at missing cycles, invalid context hashes, empty unit lists, or duplicate unit ids.
cycle_context_refs	Cycles pointing at missing contexts or mismatched context hashes.
lineage_unit_refs and lineage_cardinality	Lineage events pointing at missing cycles or units, duplicate event ids, and invalid split, merge, or rename shapes.
crosswalk_unit_refs and crosswalk_weight_sum	Crosswalks pointing at missing cycles, context hashes, or units; duplicate rows; invalid rational weights; and exhaustive weights that do not sum to one.
claim_boundary_present	Missing proves or does-not-prove declarations, or a claim boundary for the wrong package id.

RCOUNT already has compatible seed material. `rcount-rhist` maps `ReportingUnitLineage` records to RHIST `LineageEvent` records with `map_lineage_event` and `map_package_lineage`; missing cycle effective dates raise `RcountRhistError::MissingEffectiveDate`. On the consumer side, `RhistReference` lets an RCOUNT package declare a RHIST package hash, package path, cycle ids, role, and note. The RCOUNT verifier accepts only `unit-lineage`, `aggregation-crosswalk`, or `context-lineage` roles and reports `rhist_reference_declared`; the synthetic base package references `docs/fixtures/rhist/l2-three-cycle` through `SYN_RHIST_L2_PACK`

4 Fixtures

The L0 positive fixture is `l0-rename`. It is a one-event package whose `LineageEventKind::Rename` must be one-to-one. The core test `l0_rename_fixture_verifies` checks that the lineage unit references and lineage cardinality reports are present.

`l0-missing-unit` is the stale-unit negative fixture. It declares a lineage event that references a unit not present in the relevant cycle context. `rhist-core` rejects it with `LineageMissingUnit`, and `rhist-io` surfaces the same core error when loading the directory.

`l1-split-merge` is the split/merge positive package. It contains two lineage events and four crosswalk rows. The verifier checks both `crosswalk_unit_refs` and `crosswalk_weight_sum`, proving that the synthetic split and merge can be carried with exhaustive rational weights.

`l1-bad-weights` is the bad-weight negative package. Its unit and lineage references are shaped like the L1 case, but the exhaustive crosswalk weights do not sum to one. The expected failure is `RhistCoreError::CrosswalkWeightSum`.

`l2-three-cycle` is the three-cycle positive package. The tests assert that it has three cycles, seven lineage events, and nine crosswalk rows, and that its event kinds include rename, split, and merge. It is also the fixture used by the RCOUNT RHIST-reference consumer test and by `SYN_RHIST_L2_PACKAGE_HASH`.

`real-ri-tract-unchanged` is the real-source pressure fixture. It has three cycles, two lineage events, and two crosswalk rows. The verifier checks source reference resolution and `rhist-io` verifies that the preserved source bytes match `sources/source-index.json`.

The hash tests cover the package substrate itself. One test confirms that `package_content_hash` starts with `sha256:`, uses the RHIST prefix, and ignores the declared hash field. Another changes a lineage explanation and requires the hash to change; a third edits the manifest hash and expects `PackageHashMismatch`.

5 Boundaries

RHIST owns reporting-unit lineage identity. It records which unit ids are in scope for each cycle, which declared lineage events connect them, which crosswalk weights bridge unit sets, and which source hashes support those records. It does not own vote totals, candidate totals, CVR rows, audit transcripts, or count reconciliation; those remain RCOUNT claims.

RHIST also does not own district assignments or plan legality. RPLAN can use RHIST when a plan assignment crosses census or administrative unit changes, but the assignment and any plan-quality or legal claim remain outside the lineage package. RHIST is a shared base dimension for other artifacts, not an ownership transfer from those artifacts.

The source boundary is equally narrow. `verify_source_files` checks that bytes in a package match the declared SHA-256 source index. It does not prove that the public archive was complete, that an official chose the correct historical source, or that every boundary change was legally valid. Those are external evidence and governance questions.

6 Conclusion

RHIST now has a reusable reporting-unit lineage package surface. The first slice defines cycles, context unit indexes, lineage events, crosswalk weights, source indexes, claim boundaries, and a canonical package content hash. The implemented verifier catches missing units, invalid split/merge/rename cardinality, bad exhaustive crosswalk weights, source-reference drift, and package-hash drift.

The result is a bridge layer. RCOUNT can map its embedded lineage records into RHIST events and can reference RHIST packages by hash; RPLAN can later use the same lineage substrate when assignments cross unit changes. Counts and plans keep their own claims, while RHIST gives both a stable way to name how reporting units changed across cycles.

References

Gio Della-Libera. Rcount overview: Reproducible election count packages. Technical report, Apportionment Research, 2026a.

Gio Della-Libera. Rctx context packages for civic evidence. Technical report, Apportionment Research, 2026b.